شرح سطر بسطر ١٤ — حسابي (19) ورمز الطالب (29)

مشروع رِزمة · Flutter · المرحلة 6 · كل سطر مع سبب كتابته بهذه الطريقة

المرحلة 6 تبني شاشتين: حسابي (مجموعة «الملف» فقط: رمز الطالب، تبديل الطالب، الرمز السري) ورمز الطالب (الرمز مع نسخ ومشاركة، وسبب وجوده، وحالة الحفظ على الإنترنت).

كل رقم ظاهر حقيقي: النجوم من مجموع `starsEarned` في المحاولات، والرمز السري «مفعّل» إن وُجدت تجزئته في الملف، وعدد النتائج المنتظرة من عدد صفوف طابور المزامنة. لا رقم منسوخ من Figma.

مجموعات «الدراسة» و«التنزيل» و«عام» في تصميم الشاشة 19 لم تُبنَ عمدًا: لا خلفية لها بعد (حجم الخط والأصوات والتنزيل تنتظر مراحلها)، ومفتاح لا يفعل شيئًا يخدع الطالب.

لا نتحقق من الرمز الحالي قبل تغييره أو إلغائه من حسابي، لأن الطالب أصلًا دخل ملفه بعد إدخاله (أو لأنه بلا رمز). هذا قرار مقصود ومذكور في ملف الشرح الموضوعي.

lib/app/tab_navigation.dart — ماذا يحدث عند ضغط تبويب

lib/app/tab_navigation.dart

الأسطر 1–2

```dart
import 'package:flutter/material.dart';
import 'package:go_router/go_router.dart';
```

ماذا يفعل؟ استيراد Material (لـ `BuildContext` و`ScaffoldMessenger`) وgo_router.

لماذا بهذه الطريقة؟ نحتاج `context.go` للانتقال.

الأسطر 4–6

```dart
import '../core/strings/ar.dart';
import '../core/widgets/bottom_nav.dart';
import 'routes.dart';
```

ماذا يفعل؟ استيراد النصوص والشريط السفلي (لتعداد `RizmaTab`) والمسارات.

لماذا بهذه الطريقة؟ الدالة تعرف المسارات فلا تعرفها الشاشات؛ الشاشة تقول فقط أي تبويب اختير.

الأسطر 8–9

```dart
/// ما يحدث عند ضغط تبويب في الشريط السفلي. شاشات «المواد» و«تقدّمي» لم تُبنَ
/// بعد، فنخبر الطالب بدل أن نتجاهل الضغطة بصمت.
```

ماذا يفعل؟ تعليق: شاشتا «المواد» و«تقدّمي» لم تُبنيا بعد.

لماذا بهذه الطريقة؟ نشرح سبب الفرع الأخير: نخبر الطالب بدل تجاهل ضغطته بصمت.

السطر 10

```
10  void onTabSelected(BuildContext context, RizmaTab current, RizmaTab target) {
```

ماذا يفعل؟ دالة `onTabSelected(context, current, target)` دالة حرة (ليست صنفًا).

لماذا بهذه الطريقة؟ منطق بلا حالة؛ تُستدعى من الرئيسية وحسابي بنفس الطريقة فلا نكرره.

السطر 11

```
11    if (target == current) return;
```

ماذا يفعل؟ إن كان التبويب المختار هو الحالي لا نفعل شيئًا.

لماذا بهذه الطريقة؟ إعادة فتح الشاشة نفسها تعيد بناءها بلا فائدة وتُفقد موضع التمرير.

الأسطر 12–14

```
12    switch (target) {
13      case RizmaTab.home:
14        context.go(AppRoutes.home);
```

ماذا يفعل؟ `switch` على الهدف: «الرئيسية» ← `context.go(home)` .

لماذا بهذه الطريقة؟ `go` يستبدل مكدس الصفحات؛ التبويبات مستوى واحد لا تتراكم فوق بعضها.

الأسطر 15–16

```
15      case RizmaTab.profile:
16        context.go(AppRoutes.account);
```

ماذا يفعل؟ «حسابي» ← `context.go(account)` .

لماذا بهذه الطريقة؟ الشاشة الفرعية (رمز الطالب) هي التي تُفتح بـ `push` .

الأسطر 17–21

```
17      case RizmaTab.subjects:
18      case RizmaTab.progress:
19        ScaffoldMessenger.of(
20          context,
21        ).showSnackBar(const SnackBar(content: Text(ArStrings.navSectionSoon)));
```

ماذا يفعل؟ «المواد» و«تقدّمي»: نعرض `SnackBar` برسالة «قيد الإعداد».

لماذا بهذه الطريقة؟ حالتان تشتركان في فرع واحد (سقوط case في Dart 3 مسموح لحالة فارغة). رسالة صادقة بدل زر ميّت.

الأسطر 22–23

```
22    }
23  }
```

ماذا يفعل؟ إغلاق `switch` والدالة.

لماذا بهذه الطريقة؟ `switch` شاملة على التعداد فيحذّرنا المحلّل لو أضفنا تبويبًا جديدًا ونسينا معالجته.

lib/features/account/account_providers.dart — مزوّدات حسابي ورمز الطالب

lib/features/account/account_providers.dart

الأسطر 1–2

```dart
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:share_plus/share_plus.dart';
```

ماذا يفعل؟ استيراد Riverpod وحزمة `share_plus`.

لماذا بهذه الطريقة؟ `share_plus` تفتح نافذة المشاركة الأصلية في النظام.

السطر 4

```dart
import '../../core/db/database_provider.dart';
```

ماذا يفعل؟ استيراد مزوّد القاعدة.

لماذا بهذه الطريقة؟ لنصل إلى `activityDao`.

الأسطر 6–9

```dart
/// عدد النتائج المحفوظة على الجهاز وتنتظر الإرسال (طابور المزامنة).
final pendingSyncCountProvider = StreamProvider.autoDispose<int>((ref) {
  return ref.watch(appDatabaseProvider).activityDao.watchPendingSyncCount();
});
```

ماذا يفعل؟ `pendingSyncCountProvider`: بث بعدد صفوف طابور المزامنة.

لماذا بهذه الطريقة؟ `StreamProvider` فيتحدّث الرقم فور إضافة نتيجة جديدة بلا إعادة فتح الشاشة. `autoDispose` ينظّفه عند مغادرتها.

الأسطر 11–12

```dart
/// يشارك نصًّا عبر نافذة المشاركة في النظام.
typedef ShareText = Future<void> Function(String text);
```

ماذا يفعل؟ `typedef ShareText`: دالة تأخذ نصًّا وتعيد `Future<void>`.

لماذا بهذه الطريقة؟ نخفي حزمة المشاركة خلف توقيع بسيط: الشاشة لا تعرف `SharePlus` أصلًا.

الأسطر 14–19

```dart
/// منفصل كمزوّد ليُستبدل في الاختبارات (نافذة النظام لا تعمل فيها).
final shareTextProvider = Provider<ShareText>((ref) {
  return (text) async {
    await SharePlus.instance.share(ShareParams(text: text));
  };
});
```

ماذا يفعل؟ `shareTextProvider`: يعيد دالة تستدعي `SharePlus.instance.share(ShareParams(text: ...))`.

لماذا بهذه الطريقة؟ مزوّد منفصل ليُستبدل في الاختبارات (نافذة النظام لا تعمل فيها) ونتحقق أن النص المرسل صحيح.

lib/features/account/pin_manage_sheet.dart — ورقة تغيير الرمز السري أو إلغائه

lib/features/account/pin_manage_sheet.dart

الأسطر 1–6

```dart
import 'package:flutter/material.dart';

import '../../core/strings/ar.dart';
import '../../core/theme/app_colors.dart';
import '../../core/theme/app_text_styles.dart';
import '../../core/widgets/icon_tile.dart';
```

ماذا يفعل؟ استيراد Material والنصوص والألوان والأنماط ومربّع الأيقونة.

لماذا بهذه الطريقة؟ ما تحتاجه الورقة.

السطر 8

```dart
enum PinManageAction { change, disable }
```

ماذا يفعل؟ تعداد `PinManageAction { change, disable }`.

لماذا بهذه الطريقة؟ نعيد اختيار الطالب كقيمة مسمّاة لا كنص أو رقم، فيفحصها `switch` شاملة في الشاشة.

الأسطر 10–12

```dart
/// ورقة سفلية لمن لديه رمز سري مفعّل: تغييره أو إلغاؤه.
/// تعيد null إن أغلقها الطالب دون اختيار.
Future<PinManageAction?> showPinManageSheet(BuildContext context) {
```

ماذا يفعل؟ تعليق ودالة `showPinManageSheet` تعيد `Future<PinManageAction?>`.

لماذا بهذه الطريقة؟ `null` تعني أن الطالب أغلق الورقة دون اختيار؛ الشاشة تتجاهلها.

الأسطر 13–18

```dart
  return showModalBottomSheet<PinManageAction>(
    context: context,
    backgroundColor: AppColors.ground,
    shape: const RoundedRectangleBorder(
      borderRadius: BorderRadius.vertical(top: Radius.circular(28)),
    ),
```

ماذا يفعل؟ `showModalBottomSheet` بخلفية `ground` وزوايا علوية مدوّرة 28.

لماذا بهذه الطريقة؟ نفس شكل ورقتي الرمز السري و«نسيت الرمز؟» فتبدو كلها من عائلة واحدة.

الأسطر 19–21

```dart
    builder: (context) => SafeArea(
      child: Padding(
        padding: const EdgeInsetsDirectional.fromSTEB(20, 24, 20, 16),
```

ماذا يفعل؟ `SafeArea` ثم حشوة 20/24/20/16 (`Directional`).

لماذا بهذه الطريقة؟ تمنع تداخل الورقة مع حافة الجهاز السفلية.

الأسطر 22–26

```
      child: Column(
        mainAxisSize: MainAxisSize.min,
        crossAxisAlignment: CrossAxisAlignment.stretch,
        spacing: 8,
        children: [
```

ماذا يفعل؟ `Column` بأقل حجم ممكن ومسافة 8 بين العناصر.

لماذا بهذه الطريقة؟ `mainAxisSize.min` فتأخذ الورقة طول محتواها فقط لا الشاشة كلها.

السطر 27

```
          Text(ArStrings.pinManageTitle, style: AppTextStyles.section18),
```

ماذا يفعل؟ عنوان الورقة.

لماذا بهذه الطريقة؟ بنمط `section18` كباقي العناوين.

الأسطر 28–34

```
          _ActionRow(
            icon: Icons.edit_outlined,
            iconColor: AppColors.teal,
            tint: AppColors.tealTint,
            label: ArStrings.pinChange,
            onTap: () => Navigator.of(context).pop(PinManageAction.change),
          ),
```

ماذا يفعل؟ صف «تغيير الرمز»: أيقونة تعديل خضراء، وعند الضغط `pop(change)`.

لماذا بهذه الطريقة؟ `Navigator.pop` بقيمة تُعاد لمن فتح الورقة.

الأسطر 35–41

```
          _ActionRow(
            icon: Icons.lock_open_outlined,
            iconColor: AppColors.coral,
            tint: AppColors.coralTint,
            label: ArStrings.pinDisable,
            onTap: () => Navigator.of(context).pop(PinManageAction.disable),
          ),
```

ماذا يفعل؟ صف «إلغاء الرمز السري»: أيقونة قفل مفتوح مرجانية، وعند الضغط `pop(disable)`.

لماذا بهذه الطريقة؟ اللون المرجاني يلمّح أنه إجراء يُضعف الحماية.

الأسطر 42–47

```
          ],
        ),
      ),
    ),
  );
}
```

ماذا يفعل؟ إغلاق الأقواس.

لماذا بهذه الطريقة؟ نهاية الدالة.

الأسطر 49–56

```
class _ActionRow extends StatelessWidget {
  const _ActionRow({
    required this.icon,
    required this.iconColor,
    required this.tint,
    required this.label,
    required this.onTap,
  });
```

ماذا يفعل؟ صنف خاص `_ActionRow` ومُنشئه بخمسة معاملات.

لماذا بهذه الطريقة؟ ودجت صغير خاص بهذا الملف لأن لا أحد غيره يحتاجه.

الأسطر 58–62

```
  final IconData icon;
  final Color iconColor;
  final Color tint;
  final String label;
  final VoidCallback onTap;
```

ماذا يفعل؟ الحقول: الأيقونة ولونها وخلفيتها والعنوان ودالة الضغط.

لماذا بهذه الطريقة؟ غير قابلة للتغيير.

الأسطر 64–66

```
  @override
  Widget build(BuildContext context) {
    return Material(
```

ماذا يفعل؟ `build` ويعيد `Material` ببطاقة بيضاء.

لماذا بهذه الطريقة؟ `Material` ضرورية ليظهر أثر اللمس (`InkWell`) فوق اللون.

الأسطر 67–69

```
67        color: AppColors.card,
68        borderRadius: BorderRadius.circular(18),
69        clipBehavior: Clip.antiAlias,
```

ماذا يفعل؟ لون البطاقة وزوايا 18 و`clipBehavior` ليُقصّ أثر اللمس بالزوايا.

لماذا بهذه الطريقة؟ بدون القص يخرج الموج عن الزوايا المدوّرة.

الأسطر 70–74

```
70        child: InkWell(
71          onTap: onTap,
72          child: Padding(
73            padding: const EdgeInsets.symmetric(horizontal: 14, vertical: 10),
74            child: Row(
```

ماذا يفعل؟ `InkWell` للضغط، وحشوة أفقية 14 ورأسية 10، و`Row` بمسافة 12.

لماذا بهذه الطريقة؟ أول عنصر في `Row` على اليمين (RTL): مربّع الأيقونة.

الأسطر 75–76

```
75              spacing: 12,
76              children: [
```

ماذا يفعل؟ `spacing: 12` و`children`.

لماذا بهذه الطريقة؟ مسافة ثابتة بدل `SizedBox` بين كل عنصرين.

الأسطر 77–84

```
77                IconTile(
78                  iconData: icon,
79                  iconColor: iconColor,
80                  color: tint,
81                  size: 40,
82                  iconSize: 20,
83                  radius: 12,
84                ),
```

ماذا يفعل؟ `IconTile` بحجم 40 وأيقونة 20 وزوايا 12.

لماذا بهذه الطريقة؟ نفس مقاسات صفوف الإعداد في التصميم.

السطر 85

```
85                Expanded(child: Text(label, style: AppTextStyles.body15)),
```

ماذا يفعل؟ `Expanded` فيه نص العنوان.

لماذا بهذه الطريقة؟ ليأخذ العرض المتبقي ولا يتجاوز الحافة إن طال.

الأسطر 86–92

```
            ],
          ),
        ),
      ),
    );
  }
}
```

ماذا يفعل؟ إغلاق `Row` و`Padding` و`InkWell` و`Material` والدالة والصنف.

لماذا بهذه الطريقة؟ نهاية الملف.

lib/features/account/account_screen.dart — الشاشة 19: حسابي

lib/features/account/account_screen.dart

الأسطر 1–4

```
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:flutter_svg/flutter_svg.dart';
import 'package:go_router/go_router.dart';
```

ماذا يفعل؟ استيراد Material وRiverpod وflutter_svg (لسهم الصف) وgo_router.

لماذا بهذه الطريقة؟ نستعمل `context.push/go` للتنقل.

الأسطر 6–7

```
import '../../app/routes.dart';
import '../../app/tab_navigation.dart';
```

ماذا يفعل؟ استيراد المسارات ودالة التنقل بين التبويبات.

لماذا بهذه الطريقة؟ الشاشة تعرف أسماء المسارات لا الراوتر.

الأسطر 8–13

```
import '../../core/constants/app_assets.dart';
import '../../core/db/app_database.dart';
import '../../core/strings/ar.dart';
import '../../core/theme/app_colors.dart';
import '../../core/theme/app_text_styles.dart';
import '../../core/theme/profile_palette.dart';
```

ماذا يفعل؟ استيراد مسارات الأصول وأنواع القاعدة والنصوص والألوان والأنماط وألوان الشخصية.

لماذا بهذه الطريقة؟ `app_database` لنوع `Profile`.

الأسطر 14–17

```
import '../../core/widgets/bottom_nav.dart';
import '../../core/widgets/icon_tile.dart';
import '../../core/widgets/mascot.dart';
import '../../core/widgets/rizma_card.dart';
```

ماذا يفعل؟ استيراد الشريط السفلي ومربّع الأيقونة والشخصية والبطاقة.

لماذا بهذه الطريقة؟ مكوّنات مشتركة موجودة سلفًا.

الأسطر 18–21

```
import '../profiles/active_profile_provider.dart';
import '../profiles/pin_setup_sheet.dart';
import '../profiles/profile_providers.dart';
import 'pin_manage_sheet.dart';
```

ماذا يفعل؟ استيراد الملف النشط وورقة إعداد الرمز السري ومزوّدات الملفات وورقة إدارة الرمز.

لماذا بهذه الطريقة؟ نعيد استعمال ورقة إعداد الرمز التي بُنيت للمرحلة 3 فلا نكرر لوحة الأرقام.

الأسطر 23–26

```
/// الشاشة 19 «حسابي» — مجموعة «الملف» فقط (رمز الطالب، تبديل الطالب، الرمز
/// السري). مجموعات الدراسة والتنزيل والعام تنتظر مراحلها.
class AccountScreen extends ConsumerWidget {
  const AccountScreen({super.key});
```

ماذا يفعل؟ تعليق وصنف `AccountScreen` من نوع `ConsumerWidget` .

لماذا بهذه الطريقة؟ التعليق صريح أن مجموعات الدراسة والتنزيل والعام لم تُبنَ. لا نحتفظ بحالة محلية: كل شيء من مزوّدات.

الأسطر 28–32

```
  void _note(BuildContext context, String message) {
    ScaffoldMessenger.of(
      context,
    ).showSnackBar(SnackBar(content: Text(message)));
  }
```

ماذا يفعل؟ `_note` : يعرض `SnackBar` بنص.

لماذا بهذه الطريقة؟ دالة واحدة للرسائل الثلاث (تفعيل/تغيير/إلغاء).

الأسطر 34–38

```
  Future<void> _onPinTap(
    BuildContext context,
    WidgetRef ref,
    Profile profile,
  ) async {
```

ماذا يفعل؟ `_onPinTap(context, ref, profile)` : دالة غير متزامنة تُدير ضغط صف الرمز السري.

لماذا بهذه الطريقة؟ تأخذ `ref` لتقرأ المستودع عند الحاجة فقط (`read` لا `watch` لأننا في معالج ضغط).

الأسطر 39–40

```
39      final repo = ref.read(profileRepositoryProvider);
40      final enabled = profile.pinHash != null;
```

ماذا يفعل؟ نقرأ مستودع الملفات ونعرف هل الرمز مفعّل (`pinHash != null`).

لماذا بهذه الطريقة؟ الحالة من البيانات لا من متغيّر منفصل قد يختلف عنها.

الأسطر 42–48

```
42      if (!enabled) {
43        final pin = await showPinSetupSheet(context);
44        if (pin == null) return;
45        await repo.setPin(profile.id, pin);
46        if (context.mounted) _note(context, ArStrings.pinEnabledNote);
47        return;
48      }
```

ماذا يفعل؟ إن لم يكن مفعّلًا: نفتح ورقة إعداد الرمز؛ إن أغلقها الطالب (`null`) نخرج، وإلا نحفظه ونعرض «تم تفعيل الرمز السري».

لماذا بهذه الطريقة؟ الرمز يُجزّأ داخل `setPin` (SHA-256 مع salt) فلا يصل نصًّا إلى القاعدة. `context.mounted` تحمي من استعمال `context` بعد انتهاء الشاشة أثناء الانتظار.

الأسطر 50–51

```
50      final action = await showPinManageSheet(context);
51      if (action == null || !context.mounted) return;
```

ماذا يفعل؟ إن كان مفعّلًا: نفتح ورقة الإدارة؛ `null` أو شاشة مغلقة ← نخرج.

لماذا بهذه الطريقة؟ نتحقق من `mounted` مرة ثانية لأن فتح ورقة سفلية عملية انتظار.

الأسطر 52–57

```
52      switch (action) {
53        case PinManageAction.change:
54          final pin = await showPinSetupSheet(context);
55          if (pin == null) return;
56          await repo.setPin(profile.id, pin);
57          if (context.mounted) _note(context, ArStrings.pinChangedNote);
```

ماذا يفعل؟ `switch` على الإجراء: «تغيير» يفتح ورقة الإعداد ويحفظ الرمز الجديد ويعرض رسالة.

لماذا بهذه الطريقة؟ نفس خطوات التفعيل؛ `setPin` يولّد salt جديدًا فتتغيّر التجزئة كليًا.

الأسطر 58–61

```
58        case PinManageAction.disable:
59          await repo.clearPin(profile.id);
60          if (context.mounted) _note(context, ArStrings.pinDisabledNote);
61      }
```

ماذا يفعل؟ «إلغاء» ينادي `clearPin` ويعرض «أُلغي الرمز السري».

لماذا بهذه الطريقة؟ `clearPin` يمسح التجزئة والـ salt معًا ويصفّر عدّاد القفل.

السطر 62

```
62    }
```

ماذا يفعل؟ إغلاق `_onPinTap` .

لماذا بهذه الطريقة؟ نهاية الدالة.

الأسطر 64–67

```
64    @override
65    Widget build(BuildContext context, WidgetRef ref) {
66      final profile = ref.watch(activeProfileProvider);
67      final stars = ref.watch(starsByProfileProvider).value ?? const {};
```

ماذا يفعل؟ `build` : نراقب الملف النشط ونجوم كل الملفات (خريطة معرّف ← عدد).

لماذا بهذه الطريقة؟ `.value ?? const {}` : إن لم تصل القيمة بعد نعرض صفرًا مؤقتًا بدل خطأ. النجوم محسوبة من المحاولات ولا تُخزَّن.

الأسطر 69–72

```
69      return Scaffold(
70        body: SafeArea(
71          bottom: false,
72          child: profile.when(
```

ماذا يفعل؟ `Scaffold` و `SafeArea` بلا حماية من الأسفل ثم `profile.when` .

لماذا بهذه الطريقة؟ الشريط السفلي يعالج الأسفل بنفسه. `when` تفرّق التحميل والخطأ والبيانات.

الأسطر 73–79

```
73            loading: () => const SizedBox.shrink(),
74            error: (_, _) => Center(
75              child: Text(
76                ArStrings.accountLoadFailed,
77                style: AppTextStyles.body15,
78              ),
79            ),
```

ماذا يفعل؟ التحميل: فراغ. الخطأ: «تعذّر تحميل حسابك.» في المنتصف.

لماذا بهذه الطريقة؟ رسالة واضحة بدل شاشة بيضاء.

الأسطر 80–82

```
80            data: (p) => p == null
81                ? const SizedBox.shrink()
82                : SingleChildScrollView(
```

ماذا يفعل؟ البيانات: إن كان `null` (لا ملف نشط) فراغ، وإلا `SingleChildScrollView` .

لماذا بهذه الطريقة؟ الراوتر يحوّل للإقلاع في تلك الحالة؛ الفراغ احتياط لحظة التحويل.

السطر 83

```
83                padding: const EdgeInsetsDirectional.fromSTEB(20, 18, 20, 24),
```

ماذا يفعل؟ حشوة 20/18/20/24.

لماذا بهذه الطريقة؟ نفس هوامش الشاشات الأخرى.

الأسطر 84–87

```
84                child: Column(
85                  crossAxisAlignment: CrossAxisAlignment.stretch,
86                  spacing: 14,
87                  children: [
```

ماذا يفعل؟ `Column` ممتد بمسافة 14.

لماذا بهذه الطريقة؟ الصفحة تتمرّر إن صغرت الشاشة.

الأسطر 88–91

```
88                    Text(
89                      ArStrings.accountTitle,
90                      style: AppTextStyles.heading22,
91                    ),
```

ماذا يفعل؟ عنوان «حسابي» بنمط `heading22`.

لماذا بهذه الطريقة؟ في أعلى الشاشة كما في التصميم.

السطر 92

```
92                    _ProfileHeader(profile: p, stars: stars[p.id] ?? 0),
```

ماذا يفعل؟ رأس الملف: `_ProfileHeader` بالملف وعدد نجومه (`?? 0`).

لماذا بهذه الطريقة؟ ملف بلا محاولات غير موجود في الخريطة فنعرض 0.

الأسطر 93–99

```
93                    _ProfileGroup(
94                      profile: p,
95                      onStudentCode: () =>
96                          context.push(AppRoutes.studentCode),
97                      onSwitch: () => context.go(AppRoutes.profiles),
98                      onPin: () => _onPinTap(context, ref, p),
99                    ),
```

ماذا يفعل؟ `_ProfileGroup` بثلاث دوال: رمز الطالب (`push`)، تبديل الطالب (`go` إلى `/profiles`)، والرمز السري.

لماذا بهذه الطريقة؟ `push` لرمز الطالب ليعود زر الرجوع إلى حسابي. `go` لتبديل الطالب لأنها بداية جلسة طالب آخر لا شاشة فرعية.

الأسطر 100–104

```
                    ],
                  ),
                ),
          ),
        ),
```

ماذا يفعل؟ إغلاق `Column` و`SingleChildScrollView` و`when` و`SafeArea`.

لماذا بهذه الطريقة؟ نهاية الجسم.

الأسطر 105–108

```
        bottomNavigationBar: BottomNav(
          current: RizmaTab.profile,
          onSelected: (tab) => onTabSelected(context, RizmaTab.profile, tab),
        ),
```

ماذا يفعل؟ الشريط السفلي بالتبويب «حسابي»، والاختيار يمرّ بـ `onTabSelected`.

لماذا بهذه الطريقة؟ نفس منطق التنقل في كل الشاشات.

الأسطر 109–111

```
    );
  }
}
```

ماذا يفعل؟ إغلاق `Scaffold` و`build` والصنف.

لماذا بهذه الطريقة؟ نهاية `AccountScreen`.

الأسطر 113–117

```
class _ProfileHeader extends StatelessWidget {
  const _ProfileHeader({required this.profile, required this.stars});

  final Profile profile;
  final int stars;
```

ماذا يفعل؟ ودجت `_ProfileHeader` بمعاملين: الملف وعدد النجوم.

لماذا بهذه الطريقة؟ يفصل رأس الملف عن بقية الشاشة.

الأسطر 119–123

```
  @override
  Widget build(BuildContext context) {
    return RizmaCard(
      padding: const EdgeInsets.symmetric(horizontal: 16, vertical: 20),
      child: Column(
```

ماذا يفعل؟ `build` ويعيد `RizmaCard` بحشوة 16/20 و`Column`.

لماذا بهذه الطريقة؟ بطاقة بيضاء بحدّ رفيع كما في التصميم.

الأسطر 124–125

```
124         spacing: 4,
125         children: [
```

ماذا يفعل؟ مسافة 4 بين عناصر العمود.

لماذا بهذه الطريقة؟ مسافة صغيرة بين الاسم وسطر الصف والنجوم.

الأسطر 126–129

```
126           Container(
127             width: 100,
128             height: 100,
129             margin: const EdgeInsets.only(bottom: 8),
```

ماذا يفعل؟ حاوية 100×100 بهامش سفلي 8.

لماذا بهذه الطريقة؟ الأفاتار الكبير في التصميم 100.

الأسطر 130–134

```
130             decoration: BoxDecoration(
131               color: profile.color.avatarBackground,
132               shape: BoxShape.circle,
133             ),
134             alignment: Alignment.center,
```

ماذا يفعل؟ خلفية بلون الأفاتار الخاص بلون الطالب وشكل دائري، والمحتوى في الوسط.

لماذا بهذه الطريقة؟ تتبدّل بتبدّل لون شخصية الطالب.

الأسطر 135–136

```
135             child: Mascot(size: 76, color: profile.color),
136           ),
```

ماذا يفعل؟ الشخصية بحجم 76 بلون الطالب.

لماذا بهذه الطريقة؟ `Mascot` تلوّن الـ SVG وقت العرض فلا نحتاج خمسة ملفات.

السطر 137

```
137           Text(profile.name, style: AppTextStyles.title26),
```

ماذا يفعل؟ اسم الطالب بنمط `title26`.

لماذا بهذه الطريقة؟ الاسم من الملف لا نص ثابت.

الأسطر 138–141

```
          Text(
            ArStrings.accountSummary(profile.grade, stars),
            style: AppTextStyles.label14,
          ),
```

ماذا يفعل؟ سطر «الصف الخامس · 120 نجمة» عبر `accountSummary`.

لماذا بهذه الطريقة؟ دالة نصوص تعيد استعمال صيغة الجمع العربية للنجوم.

الأسطر 142–146

```
        ],
      ),
    );
  }
}
```

ماذا يفعل؟ إغلاق العناصر والصنف.

لماذا بهذه الطريقة؟ نهاية `_ProfileHeader`.

الأسطر 148–154

```
class _ProfileGroup extends StatelessWidget {
  const _ProfileGroup({
    required this.profile,
    required this.onStudentCode,
    required this.onSwitch,
    required this.onPin,
  });
```

ماذا يفعل؟ ودجت `_ProfileGroup` ومعاملاته: الملف وثلاث دوال ضغط.

لماذا بهذه الطريقة؟ الودجت لا يعرف التنقل؛ يستقبله دوالّ جاهزة (قابل للاختبار).

الأسطر 156–159

```
  final Profile profile;
  final VoidCallback onStudentCode;
  final VoidCallback onSwitch;
  final VoidCallback onPin;
```

ماذا يفعل؟ الحقول.

لماذا بهذه الطريقة؟ غير قابلة للتغيير.

الأسطر 161–163

```
  @override
  Widget build(BuildContext context) {
    final pinOn = profile.pinHash != null;
```

ماذا يفعل؟ `build` ونحسب `pinOn = pinHash != null`.

لماذا بهذه الطريقة؟ مصدر الحقيقة الوحيد لحالة الرمز.

الأسطر 165–169

```
165     return RizmaCard(
166       padding: const EdgeInsetsDirectional.fromSTEB(16, 14, 16, 4),
167       child: Column(
168         crossAxisAlignment: CrossAxisAlignment.stretch,
169         children: [
```

ماذا يفعل؟ `RizmaCard` بحشوة 16/14/16/4 و `Column` ممتد.

لماذا بهذه الطريقة؟ الحشوة السفلية 4 لأن خط الصف الأخير يضيف فراغًا.

الأسطر 170–176

```
170           Padding(
171             padding: const EdgeInsets.only(bottom: 4),
172             child: Text(
173               ArStrings.accountProfileGroup,
174               style: AppTextStyles.section18,
175             ),
176           ),
```

ماذا يفعل؟ عنوان المجموعة «الملف» بهامش سفلي 4.

لماذا بهذه الطريقة؟ كما في التصميم.

الأسطر 177–186

```
177           _SettingRow(
178             icon: Icons.vpn_key_outlined,
179             iconColor: AppColors.teal,
180             tint: AppColors.tealTint,
181             label: ArStrings.accountStudentCode,
182             value: profile.studentCode,
183             valueColor: AppColors.teal,
184             valueLtr: true,
185             onTap: onStudentCode,
186           ),
```

ماذا يفعل؟ صف «رمز الطالب»: أيقونة مفتاح خضراء، القيمة هي الرمز نفسه بالإنجليزية (`valueLtr`) بلون أخضر، وعند الضغط يفتح شاشة 29.

لماذا بهذه الطريقة؟ `valueLtr` لأن `RZ-7K4M` يجب أن يُقرأ من اليسار حتى داخل واجهة RTL وإلا انعكس ترتيب أجزائه.

الأسطر 187–193

```
187           _SettingRow(
188             icon: Icons.group_outlined,
189             iconColor: AppColors.blue,
190             tint: AppColors.blueTint,
191             label: ArStrings.accountSwitch,
192             onTap: onSwitch,
193           ),
```

ماذا يفعل؟ صف «تبديل الطالب»: أيقونة مستخدمين زرقاء بلا قيمة.

لماذا بهذه الطريقة؟ يفتح شاشة اختيار الملف.

الأسطر 194–202

```
          _SettingRow(
            icon: Icons.lock_outline,
            iconColor: AppColors.purple,
            tint: AppColors.purpleTint,
            label: ArStrings.accountPin,
            value: pinOn ? ArStrings.accountPinOn : ArStrings.accountPinOff,
            valueColor: pinOn ? AppColors.teal : AppColors.soft,
            onTap: onPin,
          ),
```

ماذا يفعل؟ صف «الرمز السري»: أيقونة قفل بنفسجية، والقيمة «مفعّل» (أخضر) أو «غير مفعّل» (رمادي).

لماذا بهذه الطريقة؟ اللون يعطي إشارة سريعة للحالة.

الأسطر 203–207

```
        ],
      ),
    );
  }
}
```

ماذا يفعل؟ إغلاق `Column` والبطاقة والصنف.

لماذا بهذه الطريقة؟ نهاية `_ProfileGroup` .

الأسطر 209–220

```
/// صف إعداد: أيقونة ملوّنة، عنوان، قيمة اختيارية، وسهم. خط سفلي رفيع بين الصفوف.
class _SettingRow extends StatelessWidget {
  const _SettingRow({
    required this.icon,
    required this.iconColor,
    required this.tint,
    required this.label,
    required this.onTap,
    this.value,
    this.valueColor = AppColors.teal,
    this.valueLtr = false,
  });
```

ماذا يفعل؟ تعليق وصنف `_SettingRow` ومعاملاته: الأيقونة ولونها وخلفيتها والعنوان والضغط، والقيمة (اختيارية) ولونها واتجاهها.

لماذا بهذه الطريقة؟ صف واحد يخدم الصفوف الثلاثة بدل تكرار الكود. القيمة الافتراضية للون أخضر.

الأسطر 222–229

```
  final IconData icon;
  final Color iconColor;
  final Color tint;
  final String label;
  final String? value;
  final Color valueColor;
  final bool valueLtr;
  final VoidCallback onTap;
```

ماذا يفعل؟ الحقول.

لماذا بهذه الطريقة؟ غير قابلة للتغيير.

الأسطر 231–236

```
  @override
  Widget build(BuildContext context) {
    return Semantics(
      button: true,
      label: value == null ? label : '$label، $value',
      excludeSemantics: true,
```

ماذا يفعل؟ `Semantics` : `build` للزر بوصف «العنوان، القيمة» ويستبعد دلالات الأبناء.

لماذا بهذه الطريقة؟ قارئ الشاشة يقرأ «الرمز السري، مفعّل» جملة واحدة بدل ثلاث عناصر منفصلة.

الأسطر 237–238

```
      child: InkWell(
        onTap: onTap,
```

ماذا يفعل؟ `InkWell` للضغط.

لماذا بهذه الطريقة؟ أثر لمس بصري.

الأسطر 239–243

```
        child: Container(
          constraints: const BoxConstraints(minHeight: 56),
          decoration: const BoxDecoration(
            border: Border(bottom: BorderSide(color: AppColors.line)),
          ),
```

ماذا يفعل؟ حاوية بحد أدنى للارتفاع 56 وخط سفلي رفيع.

لماذا بهذه الطريقة؟ 56 كما في التصميم؛ الخط يفصل الصفوف.

الأسطر 244–246

```
          child: Row(
            spacing: 12,
            children: [
```

ماذا يفعل؟ `Row` بمسافة 12: أول عنصر على اليمين.

لماذا بهذه الطريقة؟ RTL: الأيقونة أولًا ثم العنوان.

الأسطر 247–254

```
247                IconTile(
248                  iconData: icon,
249                  iconColor: iconColor,
250                  color: tint,
251                  size: 40,
252                  iconSize: 20,
253                  radius: 12,
254                ),
```

ماذا يفعل؟ مربّع الأيقونة 40.

لماذا بهذه الطريقة؟ نفس مقاسات باقي الشاشات.

السطر 255

```
255                Expanded(child: Text(label, style: AppTextStyles.body15)),
```

ماذا يفعل؟ العنوان في `Expanded`.

لماذا بهذه الطريقة؟ يدفع القيمة والسهم إلى اليسار.

الأسطر 256–261

```
256                if (value != null)
257                  Text(
258                    value!,
259                    textDirection: valueLtr ? TextDirection.ltr : null,
260                    style: AppTextStyles.label14.copyWith(color: valueColor),
261                  ),
```

ماذا يفعل؟ إن وُجدت قيمة نعرضها بلونها، وباتجاه LTR إن لزم.

لماذا بهذه الطريقة؟ الشرط داخل القائمة (`if`) بلا حاجة لودجت إضافي.

السطر 262

```
262                SvgPicture.asset(AppAssets.fwdRow, width: 18, height: 18),
```

ماذا يفعل؟ سهم الصف بحجم 18.

لماذا بهذه الطريقة؟ السهم المرسوم في Figma يشير لليسار = «إلى الأمام» في RTL.

الأسطر 263–269

```
263              ],
264            ),
265          ),
266        ),
267      );
268    }
269  }
```

ماذا يفعل؟ إغلاق العناصر والصنف.

لماذا بهذه الطريقة؟ نهاية الملف.

lib/features/account/student_code_screen.dart — الشاشة 29: رمز الطالب

lib/features/account/student_code_screen.dart

الأسطر 1–4

```dart
import 'package:flutter/material.dart';
import 'package:flutter/services.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:go_router/go_router.dart';
```

ماذا يفعل؟ استيراد Material و`services` (للحافظة) وRiverpod وgo_router.

لماذا بهذه الطريقة؟ `Clipboard` تأتي من `services`.

الأسطر 6–15

```dart
import '../../app/routes.dart';
import '../../core/db/app_database.dart';
import '../../core/strings/ar.dart';
import '../../core/theme/app_colors.dart';
import '../../core/theme/app_text_styles.dart';
import '../../core/widgets/icon_tile.dart';
import '../../core/widgets/rizma_card.dart';
import '../../core/widgets/screen_top_bar.dart';
import '../profiles/active_profile_provider.dart';
import 'account_providers.dart';
```

ماذا يفعل؟ استيراد المسارات والقاعدة والنصوص والألوان والأنماط والمكوّنات والملف النشط ومزوّدات الحساب.

لماذا بهذه الطريقة؟ كلها موجودة سلفًا أو أُنشئت في هذه المرحلة.

الأسطر 17–20

```dart
/// الشاشة 29 «رمز الطالب»: الرمز مع نسخ ومشاركة، وسبب وجوده، وحالة الحفظ على
/// الإنترنت. مجموعة الصف تظهر فقط إن أدخل الطالب رمز صف.
class StudentCodeScreen extends ConsumerWidget {
  const StudentCodeScreen({super.key});
```

ماذا يفعل؟ تعليق وصنف `StudentCodeScreen` من نوع `ConsumerWidget`.

لماذا بهذه الطريقة؟ التعليق يذكر أن مجموعة الصف مشروطة بوجود رمز صف.

الأسطر 22–24

```dart
  void _back(BuildContext context) {
    context.canPop() ? context.pop() : context.go(AppRoutes.account);
  }
```

ماذا يفعل؟ `_back`: إن أمكن الرجوع (`canPop`) نرجع، وإلا نذهب إلى حسابي.

لماذا بهذه الطريقة؟ لو فُتحت الشاشة مباشرة (رابط) لا يوجد مكدس للرجوع إليه فلا نترك الطالب عالقًا.

الأسطر 26–32

```
26    Future<void> _copy(BuildContext context, String code) async {
27      await Clipboard.setData(ClipboardData(text: code));
28      if (!context.mounted) return;
29      ScaffoldMessenger.of(
30        context,
31      ).showSnackBar(const SnackBar(content: Text(ArStrings.codeCopied)));
32    }
```

ماذا يفعل؟ `_copy`: ننسخ الرمز للحافظة ثم نعرض «تم نسخ الرمز».

لماذا بهذه الطريقة؟ نتحقق من `mounted` بعد الانتظار قبل استعمال `context`. التأكيد البصري ضروري لأن النسخ لا يظهر شيئًا بنفسه.

الأسطر 34–37

```
34    @override
35    Widget build(BuildContext context, WidgetRef ref) {
36      final profile = ref.watch(activeProfileProvider).value;
37      final pending = ref.watch(pendingSyncCountProvider).value ?? 0;
```

ماذا يفعل؟ `build`: نراقب الملف النشط وعدد النتائج المنتظرة (`?? 0` إلى أن يصل).

لماذا بهذه الطريقة؟ عدد النتائج من طابور المزامنة الحقيقي.

الأسطر 39–43

```
39      return Scaffold(
40        body: SafeArea(
41          child: profile == null
42              ? const SizedBox.shrink()
43              : SingleChildScrollView(
```

ماذا يفعل؟ `Scaffold` و`SafeArea` ثم: بلا ملف نشط فراغ، وإلا `SingleChildScrollView`.

لماذا بهذه الطريقة؟ الشاشة تتمرّر على الشاشات القصيرة.

الأسطر 44–46

```
44                child: Column(
45                  crossAxisAlignment: CrossAxisAlignment.stretch,
46                  children: [
```

ماذا يفعل؟ `Column` ممتد.

لماذا بهذه الطريقة؟ الشريط العلوي بعرض كامل.

الأسطر 47–51

```
47                    ScreenTopBar(
48                      title: ArStrings.codeTitle,
49                      subtitle: ArStrings.accountTitle,
50                      onBack: () => _back(context),
51                    ),
```

ماذا يفعل؟ `ScreenTopBar`: العنوان «رمز الطالب» والعنوان الفرعي «حسابي» وزر الرجوع.

لماذا بهذه الطريقة؟ مكوّن مشترك مع شاشات الإنشاء.

الأسطر 52–58

```
                    Padding(
                      padding: const EdgeInsetsDirectional.fromSTEB(
                        20,
                        12,
                        20,
                        24,
                      ),
```

ماذا يفعل؟ حشوة 20/12/20/24 حول المحتوى.

لماذا بهذه الطريقة؟ هوامش التصميم.

الأسطر 59–62

```
                      child: Column(
                        crossAxisAlignment: CrossAxisAlignment.stretch,
                        spacing: 14,
                        children: [
```

ماذا يفعل؟ `Column` بمسافة 14 وممتد.

لماذا بهذه الطريقة؟ تباعد البطاقات.

الأسطر 63–69

```
                          _CodeCard(
                            code: profile.studentCode,
                            onCopy: () => _copy(context, profile.studentCode),
                            onShare: () => ref.read(shareTextProvider)(
                              ArStrings.codeShareMessage(profile.studentCode),
                            ),
                          ),
```

ماذا يفعل؟ `_CodeCard` بالرمز ودالتي النسخ والمشاركة؛ المشاركة تستدعي `shareTextProvider` برسالة تحوي الرمز.

لماذا بهذه الطريقة؟ `ref.read` لأننا في معالج ضغط. المزوّد يُستبدل في الاختبار.

السطر 70

```
                          const _WhyCard(),
```

ماذا يفعل؟ بطاقة «لماذا هذا الرمز؟» ثابتة.

لماذا بهذه الطريقة؟ `const` لأن لا بيانات فيها.

الأسطر 71–79

```
                    _InfoRow(
                      iconData: Icons.sync_rounded,
                      iconColor: AppColors.teal,
                      tint: AppColors.tealTint,
                      title: ArStrings.codeNeverSynced,
                      caption: pending == 0
                          ? ArStrings.codeNothingPending
                          : ArStrings.codePending(pending),
                    ),
```

ماذا يفعل؟ بطاقة حالة الحفظ: العنوان «لم يُحفظ على الإنترنت بعد»، والسطر الثاني «N نتائج بانتظار الاتصال» أو «لا توجد نتائج».

لماذا بهذه الطريقة؟ صادقة: لا مزامنة بعد. وعند وجود نتائج نعرض عددها الحقيقي من الطابور.

الأسطر 80–81

```
                    if (profile.classCode != null)
                      _ClassGroupRow(profile: profile),
```

ماذا يفعل؟ بطاقة مجموعة الصف فقط إن كان `classCode != null`.

لماذا بهذه الطريقة؟ من لم يدخل رمز صف لا يرى مجموعة فارغة. `if` داخل القائمة.

السطر 82

```
                    const SizedBox(height: 4),
```

ماذا يفعل؟ مسافة 4 إضافية.

لماذا بهذه الطريقة؟ تفصل التذكير الأخير عن البطاقات.

الأسطر 83–87

```
                    Text(
                      ArStrings.codeKeepSafe,
                      textAlign: TextAlign.center,
                      style: AppTextStyles.caption13,
                    ),
```

ماذا يفعل؟ نص «احتفظ بالرمز في مكان آمن...» في الوسط بنمط `caption13`.

لماذا بهذه الطريقة؟ تذكير من التصميم.

الأسطر 97–88

```
                        ],
                      ),
                    ),
                  ],
                ),
              ),
        ),
      );
    }
  }
```

ماذا يفعل؟ إغلاق العناصر والصنف.

لماذا بهذه الطريقة؟ نهاية `StudentCodeScreen`.

الأسطر 108–99

```
class _CodeCard extends StatelessWidget {
  const _CodeCard({
    required this.code,
    required this.onCopy,
    required this.onShare,
  });

  final String code;
  final VoidCallback onCopy;
  final VoidCallback onShare;
```

ماذا يفعل؟ ودجت `_CodeCard` ومعاملاته: الرمز ودالتا النسخ والمشاركة.

لماذا بهذه الطريقة؟ يفصل البطاقة الداكنة.

الأسطر 117–110

```
  @override
  Widget build(BuildContext context) {
    return Container(
      padding: const EdgeInsets.all(20),
      decoration: BoxDecoration(
        color: AppColors.ink,
        borderRadius: BorderRadius.circular(24),
      ),
```

ماذا يفعل؟ `Container` : `build` بحشوة 20 ولون `ink` وزوايا 24.

لماذا بهذه الطريقة؟ بطاقة داكنة كما في التصميم تجعل الرمز بارزًا.

الأسطر 120–118

```
      child: Column(
        spacing: 8,
        children: [
```

ماذا يفعل؟ `Column` بمسافة 8.

لماذا بهذه الطريقة؟ ترتيب: «رمزك» ثم الرمز ثم الأزرار.

الأسطر 121–126

```
        Text(
          ArStrings.codeYours,
          style: AppTextStyles.label14.copyWith(
            color: const Color(0xFFB9CFC8),
          ),
        ),
```

ماذا يفعل؟ نص «رمزك» بلون أخضر رمادي فاتح (`B9CFC8`).

لماذا بهذه الطريقة؟ لون التصميم لنص ثانوي على الداكن.

الأسطر 127–131

```
        Text(
          code,
          textDirection: TextDirection.ltr,
          style: AppTextStyles.wordmark40.copyWith(letterSpacing: 1),
        ),
```

ماذا يفعل؟ الرمز بحجم 40 وبـ `TextDirection.ltr` وتباعد حروف 1.

لماذا بهذه الطريقة؟ LTR كي لا ينقلب `wordmark40` . `RZ-7K4M` نمط موجود بحجم 40 ولون فاتح.

الأسطر 132–135

```
        Padding(
          padding: const EdgeInsets.only(top: 6),
          child: Row(
            spacing: 10,
```

ماذا يفعل؟ حشوة علوية 6 و`Row` بمسافة 10.

لماذا بهذه الطريقة؟ يفصل الأزرار عن الرمز.

الأسطر 136–143

```
            children: [
              Expanded(
                child: _CodeButton(
                  icon: Icons.copy_rounded,
                  label: ArStrings.codeCopy,
                  onTap: onCopy,
                ),
              ),
```

ماذا يفعل؟ زر «نسخ» داخل `Expanded` .

لماذا بهذه الطريقة؟ الزران بعرض متساوٍ.

الأسطر 144–150

```
                Expanded(
                  child: _CodeButton(
                    icon: Icons.share_outlined,
                    label: ArStrings.codeShare,
                    onTap: onShare,
                  ),
                ),
```

ماذا يفعل؟ زر «مشاركة» داخل `Expanded`.

لماذا بهذه الطريقة؟ ترتيب RTL: «نسخ» يمين و«مشاركة» يسار كما في التصميم.

الأسطر 151–158

```
              ],
            ),
          ),
        ],
      ),
    );
  }
}
```

ماذا يفعل؟ إغلاق العناصر والصنف.

لماذا بهذه الطريقة؟ نهاية `_CodeCard`.

الأسطر 160–169

```
class _CodeButton extends StatelessWidget {
  const _CodeButton({
    required this.icon,
    required this.label,
    required this.onTap,
  });

  final IconData icon;
  final String label;
  final VoidCallback onTap;
```

ماذا يفعل؟ ودجت `_CodeButton` ومعاملاته: الأيقونة والعنوان والضغط.

لماذا بهذه الطريقة؟ زر صغير داكن خاص بهذه البطاقة.

الأسطر 171–176

```
  @override
  Widget build(BuildContext context) {
    return Semantics(
      button: true,
      label: label,
      excludeSemantics: true,
```

ماذا يفعل؟ `Semantics` : `build` زر بوصف العنوان.

لماذا بهذه الطريقة؟ يستبعد دلالات الأبناء فيُقرأ «نسخ» مرة واحدة.

الأسطر 177–182

```
      child: Material(
        color: const Color(0xFF173A3E),
        borderRadius: BorderRadius.circular(14),
        clipBehavior: Clip.antiAlias,
        child: InkWell(
          onTap: onTap,
```

ماذا يفعل؟ `Material` بلون `173A3E` وزوايا 14 و`InkWell`.

لماذا بهذه الطريقة؟ لون التصميم لزر على البطاقة الداكنة. `Material` ليظهر أثر اللمس.

الأسطر 183–188

```
          child: SizedBox(
            height: 44,
            child: Row(
              mainAxisAlignment: MainAxisAlignment.center,
              spacing: 8,
              children: [
```

ماذا يفعل؟ `SizedBox` ارتفاعه 44 و`Row` في الوسط بمسافة 8.

لماذا بهذه الطريقة؟ 44 هو الحد الأدنى المريح للمس.

الأسطر 189–193

```
                Icon(icon, size: 18, color: const Color(0xFF7ED3B2)),
                Text(
                  label,
                  style: AppTextStyles.label14.copyWith(color: AppColors.light),
                ),
```

ماذا يفعل؟ أيقونة خضراء فاتحة ونص بلون `light`.

لماذا بهذه الطريقة؟ تباين جيد على الخلفية الداكنة.

الأسطر 194–201

```
              ],
            ),
          ),
        ),
      ),
    );
  }
}
```

ماذا يفعل؟ إغلاق العناصر والصنف.

لماذا بهذه الطريقة؟ نهاية `_CodeButton`.

الأسطر 203–208

```
class _WhyCard extends StatelessWidget {
  const _WhyCard();

  @override
  Widget build(BuildContext context) {
    return RizmaCard(
```

ماذا يفعل؟ ودجت `_WhyCard` ثابت: `RizmaCard` بحشوة 16/14.

لماذا بهذه الطريقة؟ ثابت `const` لأن لا حالة له.

الأسطر 209–213

```
      padding: const EdgeInsets.symmetric(horizontal: 16, vertical: 14),
      child: Column(
        crossAxisAlignment: CrossAxisAlignment.stretch,
        spacing: 4,
        children: [
```

ماذا يفعل؟ `Column` ممتد بمسافة 4.

لماذا بهذه الطريقة؟ عنوان ثم ثلاثة صفوف.

السطر 214

```
          Text(ArStrings.codeWhyTitle, style: AppTextStyles.section18),
```

ماذا يفعل؟ عنوان «لماذا هذا الرمز؟».

لماذا بهذه الطريقة؟ نمط `section18` .

الأسطر 215–220

```
          const _WhyRow(
            icon: Icons.phone_android_rounded,
            iconColor: AppColors.teal,
            tint: AppColors.tealTint,
            text: ArStrings.codeWhy1,
          ),
```

ماذا يفعل؟ سبب 1: «انقل تقدّمك إلى جهاز جديد» بأيقونة هاتف خضراء.

لماذا بهذه الطريقة؟ أهم استعمال للرمز.

الأسطر 221–226

```
          const _WhyRow(
            icon: Icons.group_outlined,
            iconColor: AppColors.blue,
            tint: AppColors.blueTint,
            text: ArStrings.codeWhy2,
          ),
```

ماذا يفعل؟ سبب 2: «معلمك يضيفك إلى مجموعته» بأيقونة مستخدمين زرقاء.

لماذا بهذه الطريقة؟ يخدم رمز الصف لاحقًا.

الأسطر 227–232

```
          const _WhyRow(
            icon: Icons.shield_outlined,
            iconColor: AppColors.purple,
            tint: AppColors.purpleTint,
            text: ArStrings.codeWhy3,
          ),
```

ماذا يفعل؟ سبب 3: «لا يحتاج بريدًا ولا رقم هاتف» بأيقونة درع بنفسجية.

لماذا بهذه الطريقة؟ يؤكد خصوصية الطالب.

الأسطر 233–237

```
        ],
      ),
    );
  }
}
```

ماذا يفعل؟ إغلاق العناصر والصنف.

لماذا بهذه الطريقة؟ نهاية `_WhyCard` .

الأسطر 239–245

```
class _WhyRow extends StatelessWidget {
  const _WhyRow({
    required this.icon,
    required this.iconColor,
    required this.tint,
    required this.text,
  });
```

ماذا يفعل؟ ودجت `_WhyRow` ومعاملاته: الأيقونة ولونها وخلفيتها والنص.

لماذا بهذه الطريقة؟ صف واحد يخدم الأسباب الثلاثة.

الأسطر 247–250

```
  final IconData icon;
  final Color iconColor;
  final Color tint;
  final String text;
```

ماذا يفعل؟ الحقول.

لماذا بهذه الطريقة؟ غير قابلة للتغيير.

الأسطر 252–258

```
252   @override
253   Widget build(BuildContext context) {
254     return Padding(
255       padding: const EdgeInsets.symmetric(vertical: 8),
256       child: Row(
257         spacing: 12,
258         children: [
```

ماذا يفعل؟ `build`: حشوة رأسية 8 و`Row` بمسافة 12.

لماذا بهذه الطريقة؟ تباعد الصفوف.

الأسطر 259–266

```
259           IconTile(
260             iconData: icon,
261             iconColor: iconColor,
262             color: tint,
263             size: 40,
264             iconSize: 20,
265             radius: 12,
266           ),
```

ماذا يفعل؟ مربّع الأيقونة 40.

لماذا بهذه الطريقة؟ نفس مقاسات التصميم.

الأسطر 267–272

```
267           Expanded(child: Text(text, style: AppTextStyles.body15)),
268         ],
269       ),
270     );
271   }
272 }
```

ماذا يفعل؟ النص في `Expanded` ثم إغلاق الصنف.

لماذا بهذه الطريقة؟ النص الطويل يلتف على سطرين بدل أن يتجاوز الحافة.

الأسطر 274–277

```
274 class _ClassGroupRow extends StatelessWidget {
275   const _ClassGroupRow({required this.profile});
276
277   final Profile profile;
```

ماذا يفعل؟ ودجت `_ClassGroupRow` ومعامله: الملف.

لماذا بهذه الطريقة؟ يفصل منطق مجموعة الصف.

الأسطر 279–281

```
279   @override
280   Widget build(BuildContext context) {
281     return _InfoRow(
```

ماذا يفعل؟ `build` ويعيد `_InfoRow`.

لماذا بهذه الطريقة؟ نعيد استعمال بطاقة المعلومة نفسها.

الأسطر 282–289

```
282       iconData: Icons.group_outlined,
283       iconColor: AppColors.amberDark,
284       tint: AppColors.amberTint,
285       title: ArStrings.codeClassGroup,
286       caption: ArStrings.codeClassLine(profile.classCode!),
287     );
288   }
289 }
```

ماذا يفعل؟ أيقونة عنبرية، والعنوان «مجموعة الصف» والتوضيح «رمز الصف: GZ-5A» من `classCode`.

لماذا بهذه الطريقة؟ `!` آمنة لأننا لا نبني هذا الودجت إلا إن كان `classCode != null`. لا نخترع اسم مجموعة لا نعرفه (التصميم يكتب «الخامس (أ)» وهو مثال).

الأسطر 291–299

```
291 /// بطاقة معلومة: أيقونة ملوّنة وسطر عنوان وسطر توضيح.
292 class _InfoRow extends StatelessWidget {
293   const _InfoRow({
294     required this.iconData,
295     required this.iconColor,
296     required this.tint,
297     required this.title,
298     required this.caption,
299   });
```

ماذا يفعل؟ تعليق وصنف `_InfoRow` ومعاملاته: الأيقونة ولونها وخلفيتها والعنوان والتوضيح.

لماذا بهذه الطريقة؟ بطاقة معلومة تُستعمل للمزامنة والمجموعة.

الأسطر 301–305

```
301   final IconData iconData;
302   final Color iconColor;
303   final Color tint;
304   final String title;
305   final String caption;
```

ماذا يفعل؟ الحقول.

لماذا بهذه الطريقة؟ غير قابلة للتغيير.

الأسطر 307–314

```
307   @override
308   Widget build(BuildContext context) {
309     return DecoratedBox(
310       decoration: BoxDecoration(
311         color: AppColors.card,
312         borderRadius: BorderRadius.circular(18),
313         border: Border.all(color: AppColors.line),
314       ),
```

ماذا يفعل؟ `DecoratedBox` : `build` أبيض بزوايا 18 وحدّ رفيع.

لماذا بهذه الطريقة؟ مثل بطاقات التصميم.

الأسطر 315–319

```
315       child: Padding(
316         padding: const EdgeInsets.symmetric(horizontal: 14, vertical: 12),
317         child: Row(
318           spacing: 12,
319           children: [
```

ماذا يفعل؟ حشوة 14/12 و `Row` بمسافة 12.

لماذا بهذه الطريقة؟ تباعد الصف.

الأسطر 320–327

```
320             IconTile(
321               iconData: iconData,
322               iconColor: iconColor,
323               color: tint,
324               size: 40,
325               iconSize: 20,
326               radius: 12,
327             ),
```

ماذا يفعل؟ مربّع الأيقونة 40.

لماذا بهذه الطريقة؟ الحجم نفسه في كل مكان.

الأسطر 328–331

```
328             Expanded(
329               child: Column(
330                 crossAxisAlignment: CrossAxisAlignment.start,
331                 children: [
```

ماذا يفعل؟ `Expanded` ثم `Column` محاذاة البداية.

لماذا بهذه الطريقة؟ يأخذ العرض المتبقي.

الأسطر 332–333

```
                Text(title, style: AppTextStyles.body15),
                Text(caption, style: AppTextStyles.caption13),
```

ماذا يفعل؟ سطر العنوان بنمط `body15` وسطر التوضيح بنمط `caption13`.

لماذا بهذه الطريقة؟ تسلسل بصري واضح.

الأسطر 334–342

```
              ],
            ),
          ),
        ],
      ),
    ),
  );
 }
}
```

ماذا يفعل؟ إغلاق العناصر والصنف.

لماذا بهذه الطريقة؟ نهاية الملف.